

Analysis of Algorithms

Yihan Sun

Todos in the first week

- Join GradeScope, CampusWire and CodeForces
- **Course Policy Test on GradeScope**
 - 1 pt in the overall score
- **Make a submission to “candies” on gradescope (the score of that entry will be #candies you earn in this class)**
- **Work on the entrance exam (due Sunday 1/11)**
 - On codeforces.com

Programming assignments

- On CodeForces
- You can view the difficulty for many of them to be close to the real interview problems (actually they are easier than the interview problems, but a good start!)
- For each problem
 - Space/time limit; 10-20 test cases
- You need to submit a **report** (on gradescope, just like regular HW solutions) to explain your algorithm, and why this solves the problem
 - You also need to provide your codeforces id and submission id to help us find your submission.
 - Imagine you are explaining your algorithm to an interviewer in the report
- **You get NO POINTS if you do not submit the report!**
- **Basic programming part has deadline one week earlier!**

Q&As for programming assignments

- **Can I submit multiple times?**
 - Yes! When you submit your code to codeforces, you will get the score immediately. You can resubmit before the ddl, we will use you highest score as your grade.
- **Can I get partial points?**
 - Yes! For example, if you pass 7 out of 10 test cases, you get 70% of the points. Not all test cases have the same weight though. The test cases cover different input sizes and corner cases. Passing the samples doesn't necessary mean getting everything correct.
- **If I use languages such as Python, will the time limit be longer?**
 - We use the default time scaling on codeforces (see here: <https://codeforces.com/group/TUYaOUyb7Y/contest/506200/extraTimeFactors>)
 - However, empirically, C++ will still give you the fastest running time for the same algorithm

Q&As for programming assignments

- **Will test cases be provided?**
 - We usually provide 1-3 simple test cases to help you understand the problem. We do not provide the real test cases.
 - Designing test cases is as important as finding solutions!
 - You need to understand the corner cases and deal with them when you write the code
 - This is the same in the real-life, either when you are in an interview, or when you are working in a company, and you have no idea what the users will do ...
 - In many cases, you only see the bug, and you need to find test cases to reproduce the bug and fix it.



Why don't you provide the test cases?

A software tester walks into a bar.
Runs into a bar.
Crawls into a bar.
Dances into a bar.
Flies into a bar.
Jumps into a bar.
And orders:
a beer.
2 beers.
0 beers.
99999999 beers.
a lizard in a beer glass.
-1 beer.
"qwertyuiop" beers.
Testing complete.
A real customer walks into the bar and asks
where the bathroom is.
The bar goes up in flames.

Source:
<https://tech.forums.softwareag.com/>

(As you can see, considering all possible tests is hard but important)

Why don't you provide the test cases?

- But still, you'll see if your code fails certain test cases, and you can resubmit until it's fully correct
- And you get partial points even if it doesn't pass all tests
- If you really need help, very rarely we provide (at most) 1 test case to help you debug. This can be done only in in-person office hours. After you explain your code and discuss it with the TAs, and if they feel necessary, we will provide at most 1 test case.

PLAGIARISM WARNING



- **Cheating or plagiarism will NOT be tolerated!!!**
- **Collaboration policy (Homework)**
 - You can get help from the instructors, TAs, textbook (or relevant books), internet, or your classmates (any source **should be cited**, including collaborator's name, links to online sources, book title, etc.)
 - You CANNOT:
 - **Share/show** your written solutions / code with anyone else (except for course staffs)
 - Read other's solutions / code
 - Copy anything from other source
 - Get help from others without acknowledging them in your submission, or use any relevant resources (e.g., online article) without citing them.
 - If someone copied your solution, both of you are violating the rule and the same penalty
- **More information available on the course website**
- **See UCR academic integrity for additional information:**
 - <https://conduct.ucr.edu/policies/academic-integrity-policies-and-procedures>

Can I use AI to help me solve homework problems?...

- Yes you can!
- Just like all other resources (textbooks, Internet articles, search engines, etc.), AI tools can be used to help you solve problems. It is also important for you to learn how to use them, as part of your “problem-solving” skill
- But the goal is to use them in the best way such that your own skills are improved, i.e., it helps you figure out the solution, and you understand the solution yourself.
- **Whenever an AI tool is used, you need to cite the full conversation.**
 - Since it is usually hard to generate the same answer again, and no url can be provided, the screenshot of the full conversation is necessary.
 - It won't affect your grade if you cite properly. But it will be viewed as cheating if you don't cite properly.

What happens if I violate the rules?

- First of all, we don't want to see this.
- If this really happens, the minimum penalty for a homework assignment is that you will get **0 points for this entire assignment** (not just the affected problem), including programming, written, bonus, candies, **AND a -10 points penalty** in your final grade for **every cheating case**. This is the minimum penalty for all violations, including missing citations, not mentioning collaborators, etc.
- For more severe cases, you may receive -20 points penalty or fail the class.
- Since we will drop the lowest grade among your five assignments, this assignment will likely become your lowest-grade assignment. In most of the cases, the only penalty is -10 points, and you still have a chance to pass. This is also to warn you not to do this twice. The next time you will get a 0 point counted towards your final grade.
- **The -10 or -20 penalty is firm. There is no space to argue.**

Homework: Grace Days

- **For emergencies**

- Sickness, family issues, internet issues, or other reasons

- **4 grace days to use for the **entire** quarter, 2 days maximum each time**

- You don't lose points and you don't need any reasons
- The number of grace days used must be an integer (you can't use 0.25 grace days if you submit 6 hours late, it's still 1 grace day)
- Counted for each deadline/gradescope entry (10 ddls)

- **Plan ahead!**

- If you got sick, power outage, ..., use your grace days
- **We won't accept other excuses for late submission**, except for very, very severe issues (and you need to provide some evidence for that)
- You can't use all grace days because you want to go to Disneyland, and later you ask for more grace days because you get a fever

FAQ for Grace Days

- **Q: Can you extend the deadline? I have other deadlines or exams recently / I got sick / I'm too tired / I just need more time / there was an Internet issue before the deadline.**
 - We understand that everyone may have emergency issues at the last minute. That's why we have grace days! In this case, you can extend your deadlines in a more flexible way - since everyone has a different schedule, you may get busy and feel it useful to extend certain deadlines.
 - The reason can be that you need more time so you can get more points, and it can also be that there was a sudden Internet issue, so you missed the deadline. I would suggest you save at least one grace day for emergencies since that's not controllable by you.
 - You should still aim to submit on normal deadlines.

Exams

- There will be two midterm exams. Each will take 20% in your final score.
- To align the two sessions, both exams will be in the evening. If you have hard conflicts, we'll try our best to resolve them.
- **Why we need to have so many exams?...**
 - Yes. Exams are boring. I agree. It is almost impossible to solve the real interesting problems and challenge yourselves in such a short time.
 - However, for the teaching team, it is also very challenging to make sure we can effectively test your performance when AIs become so smart today. Based on our previous experiences, AI can do a very good job in solving the homework problems. This makes it very hard to fairly evaluate how well you learn the knowledge.
 - It is also bad for learning – after all, the easiest, greedy way of learning (and get an OK score) is to let the AI do the homework with a good score. Then even you work on exam badly, you still pass (sometimes you can even get an A- or A)

Tentative score-to-grade mapping

- **A+: >100%**
- **A: 90%**
- **A-: 85%**
- **B+/B/B-: 80%/75%/70%**
- **C/D: 65%/60%**
- **F: <60%**

Bonus Candies and Chocolate!

- **You can get bonus “candies” from**
 - Asking/answering questions in class
 - Actively asking/answering questions online (campuswire)
 - Bonus questions in homework, exams, programming problems
- **You could use it for:**
 - Get real candies in the classroom
 - **Real bonus points (up to 10pts)**
 - They will finally be mapped to some bonus points. But as bonus points, the exchange rate is based on your rank. For example, the top 1 candy winner will get 10 points, the others will be scaled accordingly. In our previous experience, 2-3 candies may be mapped to 1 point.
 - Get recommendation letters

Other suggestions

- **If you feel like you don't have solid background for the course**
 - Get help from instructors or TAs! We are happy to help!
 - Read the textbook and other books, get help from the internet (e.g., Wikipedia), review contents of previous relevant courses ...
- **Participate in the discussions and talk to TAs and your classmates!**
- **Competitive Coding Club (CCC@UCR): get more training for programming**
- **CS119L: Lab class to get training for programming**

Analyzing algorithms

- Predict how your algorithm performs in practice

- **Criteria**

- Running time
 - Space usage
 - Cache I/O
 - Disk I/O
 - Network bandwidth
 - Power consumption
 - Lines of codes
- Time complexity** ←

What are good algorithms?

- Tale about Guass



One day Gauss's teacher asked his class to add together all the numbers from 1 to 100, assuming that this task would occupy them for quite a while. He was shocked when young Gauss, after a few seconds thought, wrote down the answer 5050. (source: <https://nrich.maths.org/2478>)

Other students:

```
sum = 0;  
for (int i = 1; i <= n; i++)  
    sum += i;
```

$O(n)$ time complexity

Why we say the complexity is $O(n)$?

Guass:

sum = $(1+n)*n/2$;

$O(1)$ time complexity

$$\begin{array}{cccccccccccc} 1 & + & 2 & + & 3 & + & 4 & + & 5 & + & 6 & + & 7 & + & 8 & + & 9 & + & 10 & = & ? \\ \hline & & & & & & & & + & & & & & & & & & & & & \\ & & & & & & & & + & & & & & & & & & & & & \\ & & & & & & & & + & & & & & & & & & & & & \\ & & & & & & & & + & & & & & & & & & & & & \\ & & & & & & & & + & & & & & & & & & & & & \end{array}$$
$$(1+10) + (2+9) + (3+8) + (4+7) + (5+6) = ?$$

Analyze running time

- **Random Access Machine (RAM) model**

- Every operation, including memory access, arithmetic operations, etc., takes **unit time**

```
sum = 0;  
for (int i = 1; i <= n; i++)    sum = (1+n)*n/2;  
    sum += i;
```

$3n+2$ operations?

3 operations?

Analyze running time

- **Random Access Machine (RAM) model**

- Every operation, including memory access, arithmetic operations, etc., takes **unit time**

- **We only care about the **order of the cost**, e.g., we omit**

- Lower-order terms
- Constants

Asymptotic Analysis

- **We care about how faster a function grows for large n**

- **We usually talk about **worst-case** time complexity**

- Among all possible inputs
- Sometimes we use “average” time complexity for randomized algorithms

Would you rather have a million dollars or one cent on day one, doubled every day for a month?

- A. I want a million dollars immediately!!



- B. I want to have one cent in the first day, doubled every day for a month!



- Actually, even in February, for choice B you can get more than 1 million!
- Your total reward on day i is
 - $1 + 2 + 4 + \dots + 2^{i-1}$
 - $= 2^i - 1$ cents $= (2^i - 1)/100$ dollars
- Is it larger than 1,000,000 dollars?
 - It will be, as long as i is large enough!

Why we use “asymptotical analysis”?

- Because we only care about how fast a function grows!



The ruler liked the chess very much, and wanted to give the inventor a prize. He could have anything he wished, the ruler said. The inventor requested that

- One grain of wheat be put in the first square
- Two grains of wheat in the second square.
- Four grains of wheat in the third
-

In general, the number of grains of wheat in one square is always the double of the previous square. The ruler laughed it off as an insignificant prize for his achievement and the ruler granted his wish. Later the court treasurers reported that the entire grain reserve of the kingdom was not sufficient to fulfill the request!

Grow exponentially (2^n)!

18,446,744,073,709,551,615

This is more wheat than in the entire whole world; in fact, it would fill a building 25 miles long, 25 miles wide, and 1000 feet tall.

Growth of Functions

- The **growth** of the function is what interests us (**large data**)
 - For the cost function $f(n)$ we care when n is large enough
 - When n is small, $f(n)$ is small anyway. We care the analysis when the running time can be long
 - The **constant factors** and **lower-order terms** don't affect the growth of the function

$$f(n) = (2^n - 1)/100 \quad \text{vs.} \quad g(n) = 1,000,000$$

When n is large enough, $f(n)$ will be much larger than $g(n)$

Asymptotic notation

```
sum = 0;  
for (int i = 1; i ≤ n; i++)  
    sum += i;
```

$\text{sum} = (1+n)*n/2;$

$3n+2$ operations?

3 operations?

$O(n)$ operations

$O(1)$ operations

- OK, then what does big-O mean?

Asymptotic notations

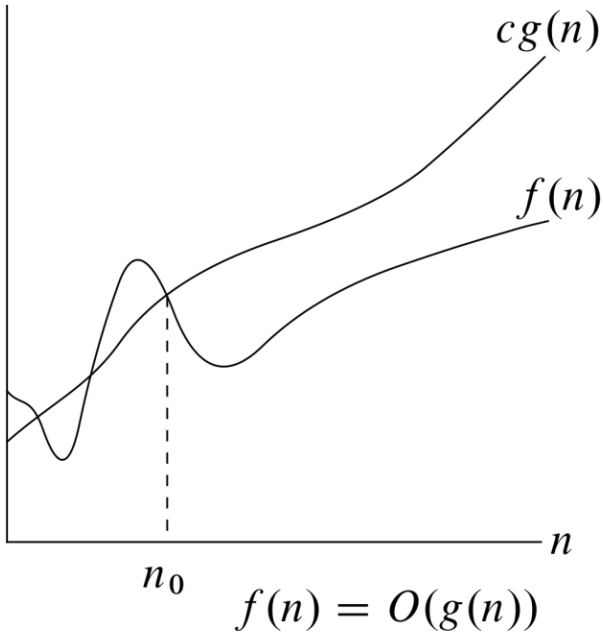
- Big-O: asymptotically **smaller than or equal to** $\leq$
larger than or equal to $\geq$
smaller than $<$
larger than $>$
equal to $=$
- n is $O(n)$
- $3n + 2$ is $O(n)$
- $\log n + \sqrt{n} + 4$ is also $O(n)$
- What happens if we want to say other cases?

Analogy to real numbers

Functions	Real numbers
$f(n) = O(g(n))$	$a \leq b$
$f(n) = \Omega(g(n))$	$a \geq b$
$f(n) = \Theta(g(n))$	$a = b$
$f(n) = o(g(n))$	$a < b$
$f(n) = \omega(g(n))$	$a > b$

O - notation (Big-O)

For some constant c , when n is large enough,
 $f(n)$ is **no more than** $cg(n)$



Assume we use $n > n_0 = 10$

$f(n) = 5n^2$, $g(n) = n^2$, We can use $c = 5$. Then $0 \leq f(n) \leq cg(n)$ always holds for $n > 10$
So $f(n) = O(g(n))$, i.e., $5n = O(n^2)$

$$f(n) = O(g(n))$$

$$\exists c > 0, n_0 > 0$$

$$0 \leq f(n) \leq cg(n)$$

$$\text{for } n \geq n_0$$

$g(n)$ is an asymptotic
upper-bound for $f(n)$

$$5n^2$$

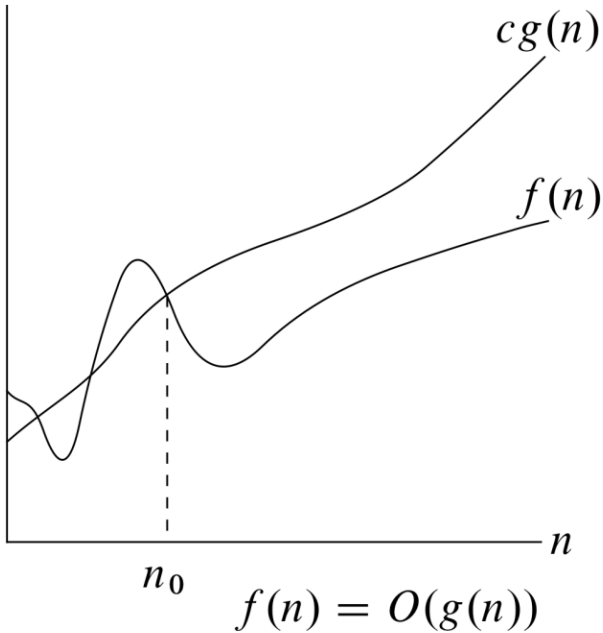


$$5n^2$$



O - notation (Big-O)

For some constant c , when n is large enough, $f(n)$ is **no more than** $cg(n)$



Assume we use $n > n_0 = 10$

$f(n) = n^2 + 2n$, $g(n) = n^3$, We can use $c = 1$. Then $0 \leq f(n) \leq g(n)$ holds for $n > 10$ so $f(n) = n^2 = O(n^3)$

$$f(n) = O(g(n))$$

$$\exists c > 0, n_0 > 0$$

$$0 \leq f(n) \leq cg(n)$$

$$\text{for } n \geq n_0$$

$g(n)$ is an asymptotic
upper-bound for $f(n)$

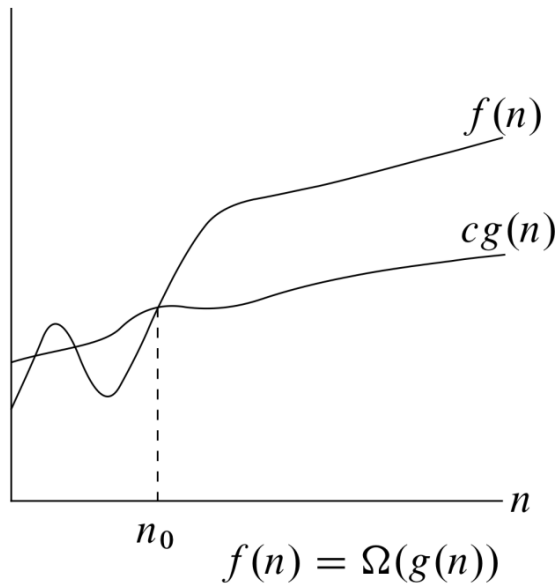
$$n^2 + 2n \quad n^3$$

↑ ↑

When $n > 10$, $n^3 > 10n^2$, but $n^2 + 2n < 10n^2$

Ω -notation

For some constant c , when n is large enough,
 $f(n)$ is **no less than** $cg(n)$



$$f(n) = \Omega(g(n))$$

$$\exists c > 0, n_0 > 0$$

$$0 \leq cg(n) \leq f(n)$$

$$\text{for } n \geq n_0$$

$g(n)$ is an asymptotic
lower-bound for $f(n)$

Assume we use $n > n_0 = 10$

$f(n) = 5n^2$, $g(n) = n^2$, We can use $c = 5$, so $f(n) = 5n^2 = \Omega(n^2)$

$f(n) = n^2 - n$, $g(n) = n$, We can use $c = 1$, so $f(n) = n^2 - n = \Omega(n)$

Θ -notation

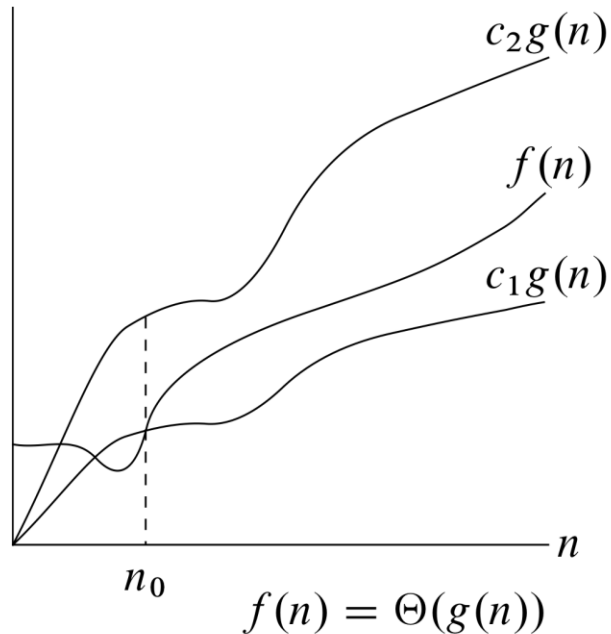
For some constant c , when n is large enough,

$f(n)$ is **no less than** $c_1 g(n)$ and **no more than** $c_2 g(n)$

$$f(n) = \Theta(g(n))$$

$$\exists c_1, c_2 > 0, n_0 > 0$$

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \\ n \geq n_0$$



$g(n)$ is an asymptotic
tight-bound for $f(n)$

Assume we use $n > n_0 = 10$

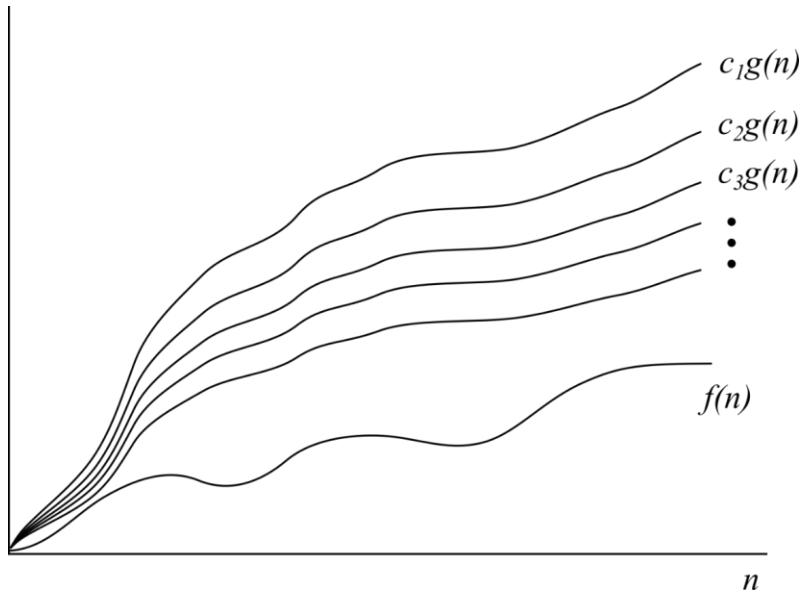
$f(n) = 5n^2, g(n) = n^2$, We can use $c_1 = c_2 = 5$, so $f(n) = 5n^2 = \Theta(n^2)$

$f(n) = n^2 + n, g(n) = n^2$, We can use $c_1 = 1, c_2 = 2$, so $f(n) = n^2 + n = \Theta(n^2)$

o-notation

$f(n) = o(g(n))$ means $f(n)$ is asymptotically smaller than $g(n)$

For any constant c , we can make n large enough, such that $f(n) < cg(n)$



$$f(n) = o(g(n))$$

$$\forall c > 0, \exists n_0 > 0 \\ 0 \leq f(n) < cg(n) \\ \text{for } n \geq n_0$$

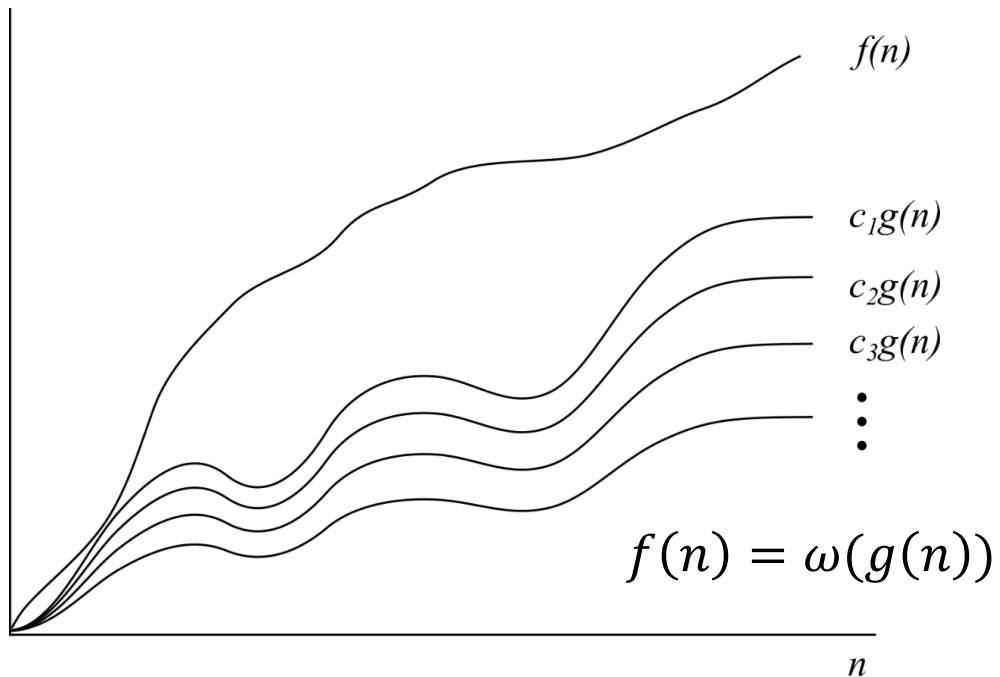
$$f(n) = o(g(n)) \\ \Leftrightarrow \\ f(n) \text{ is } O(g(n)) \text{ but} \\ f(n) \text{ is not } \Theta(g(n))$$

$f(n)$ is “strictly smaller” than $g(n)$:
For large enough n ,
whatever constant c you use,
 $f(n) < cg(n)$

ω -notation

$f(n) = \omega(g(n))$ means $f(n)$ is asymptotically larger than $g(n)$

For any constant c , we can make n large enough, such that $f(n) > cg(n)$



$$\forall c > 0, \exists n_0 > 0 \\ 0 \leq cg(n) < f(n) \\ \text{for } n \geq n_0$$

$$f(n) = o(g(n))$$

$\Leftrightarrow$

$f(n)$ is $\Omega(g(n))$ but
 $f(n)$ is not $\Theta(g(n))$

$f(n)$ is “strictly larger” than $g(n)$:
For large enough n ,
whatever constant c you use,
 $f(n) > cg(n)$

Analogy to real numbers

Functions	Real numbers
$f(n) = O(g(n))$	$a \leq b$
$f(n) = \Omega(g(n))$	$a \geq b$
$f(n) = \Theta(g(n))$	$a = b$
$f(n) = o(g(n))$	$a < b$
$f(n) = \omega(g(n))$	$a > b$

Another easy way to compare two functions

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

$$\begin{array}{ll} \bullet 0: & f(n) = o(g(n)) \\ \bullet c > 0: & f(n) = \Theta(g(n)) \\ \bullet \infty: & f(n) = \omega(g(n)) \end{array} \left. \begin{array}{l} \} \\ \} \\ \} \end{array} \right\} \begin{array}{l} O(g(n)) \\ \Omega(g(n)) \end{array}$$

$$n = \frac{1}{n}(n^2)$$

$$\lim_{n \rightarrow \infty} \frac{n}{n^2} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

$$\Rightarrow n = o(n^2)$$

Another easy way to compare two functions

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

- 0: $f(n) = o(g(n))$
- $c > 0$: $f(n) = \Theta(g(n))$
- ∞ : $f(n) = \omega(g(n))$

$\left. \begin{array}{l} \text{• } 0: f(n) = o(g(n)) \\ \text{• } c > 0: f(n) = \Theta(g(n)) \\ \text{• } \infty: f(n) = \omega(g(n)) \end{array} \right\} \begin{array}{l} O(g(n)) \\ \Omega(g(n)) \end{array}$

$$3n + \log n = __(n)$$

$$\begin{aligned} & \lim_{n \rightarrow \infty} \frac{3n + \log n}{n} \\ &= \lim_{n \rightarrow \infty} 3 + \frac{\log n}{n} \\ &= 3 \end{aligned}$$

$$\Rightarrow 3n + \log n = \Theta(n)$$

Another easy way to compare two functions

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

$$\begin{array}{ll} \bullet 0: & f(n) = o(g(n)) \\ \bullet c > 0: & f(n) = \Theta(g(n)) \\ \bullet \infty: & f(n) = \omega(g(n)) \end{array} \left. \begin{array}{l} \} \\ \} \\ \} \end{array} \right\} \begin{array}{l} O(g(n)) \\ \Omega(g(n)) \end{array}$$

$$n^2 = \text{---}(n \log n)$$

$$\begin{aligned} & \lim_{n \rightarrow \infty} \frac{n^2}{n \log n} \\ &= \lim_{n \rightarrow \infty} \frac{n}{\log n} \\ &= +\infty \end{aligned}$$

$$\Rightarrow n^2 = \omega(n \log n)$$

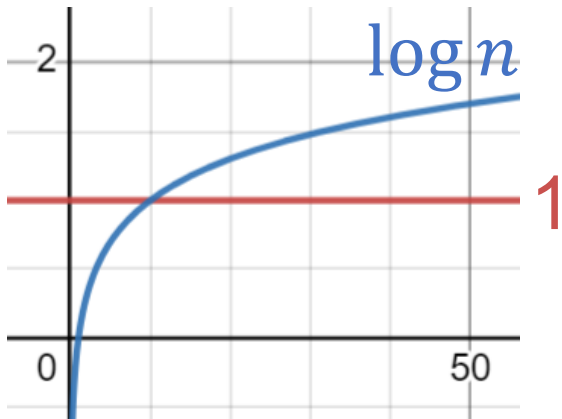
Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

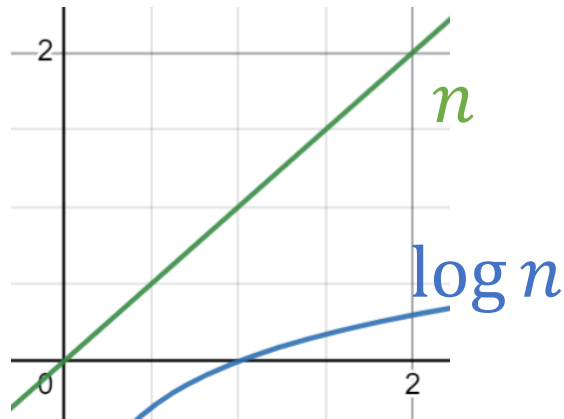
$$c < \log n$$

c ? $\log n$



$$< n$$

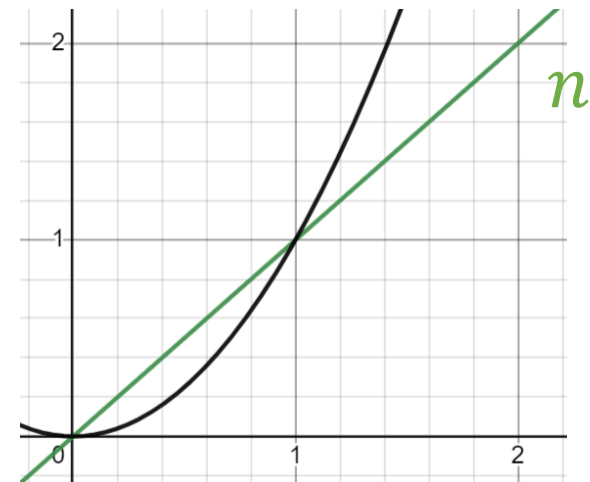
? n



$\alpha > 1$

$$< n^\alpha$$

? n^2



Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

$$c < \log \log n < \log n$$

$$< n$$

$$\alpha > 1$$

$$< n^\alpha$$

$$\log \log n \quad ? \quad \log n$$


$$T$$

$$\log T \quad ? \quad T$$

Compare two functions:
Replace some common
terms!

Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

$$c < \log \log n < \log n$$



$0 < \epsilon < 1$

$$n^\epsilon < n$$

$\alpha > 1$

$$< n^\alpha$$

$$n^{1/2} = \sqrt{n} \quad ? \quad n$$



$$T = \sqrt{n}$$

$$T \quad ? \quad T^2$$

Compare two functions:
Replace some common
terms!

Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

$$c < \log \log n < \log n$$

$0 < \epsilon < 1$

$$< n^\epsilon < n$$

$\alpha > 1$

$$< n^\alpha$$

$$\log n \text{ ? } \sqrt{n}$$

$$\lim_{n \rightarrow +\infty} \frac{\log n}{\sqrt{n}}$$

Using L'Hôpital's rule ($\frac{0}{0}$ form)

$$\begin{aligned} \text{(derivative)} \quad &= \frac{1/n}{\frac{1}{2} \cdot 1/\sqrt{n}} \\ &= \frac{2}{\sqrt{n}} = 0 \end{aligned}$$

Compare two functions:

Use L'Hôpital's rule if
computing the limit
seems hard

Commonly-used functions

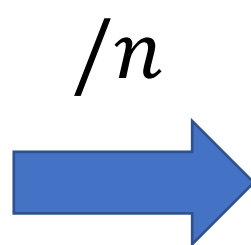
For large enough n , we have (asymptotically):

Constant $c > 0$

$\beta > 1$ $0 < \epsilon < 1$

$\alpha > 1$

$$c < \log \log n < \log n < \log^\beta n < n^\epsilon < n < n \log n < n^\alpha$$


$$\begin{array}{ccc} n \log n & ? & n^2 \\ \log n & ? & n \end{array}$$

Compare two functions:
Multiply or divide the
same term

Commonly-used functions

For large enough n , we have (asymptotically):

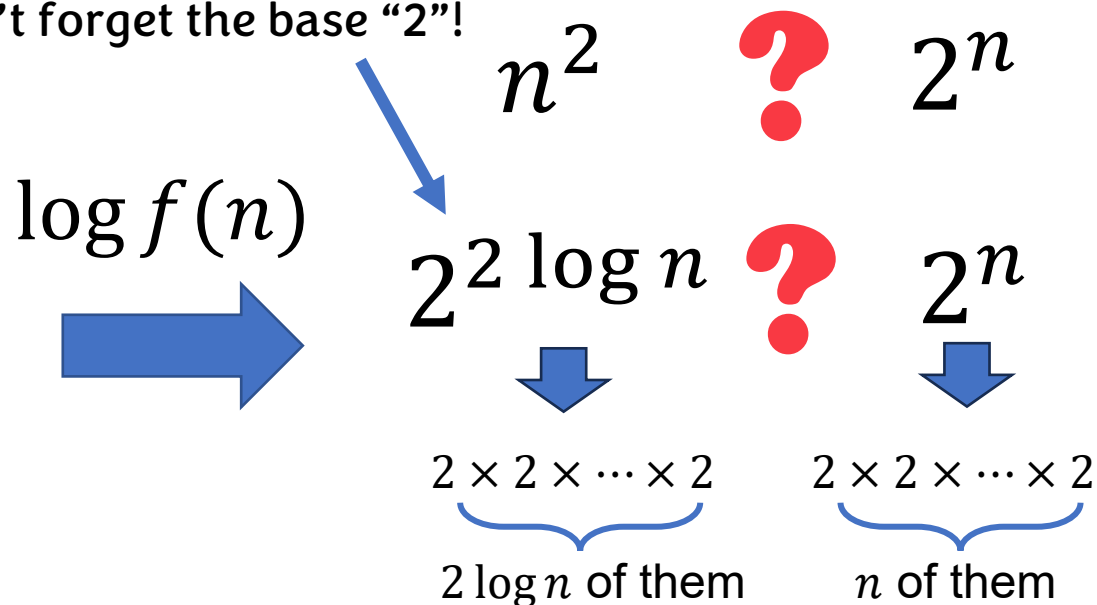
Constant $c > 0$

$\beta > 1$ $0 < \epsilon < 1$

$\alpha > 1$ $\gamma > 1$

$$c < \log \log n < \log n < \log^\beta n < n^\epsilon < n < n \log n < n^\alpha < \gamma^n$$

Don't forget the base "2"!



Compare two functions:
Apply the same (increasing) function to both f and g to transform them into other forms!

Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

$\beta > 1$ $0 < \epsilon < 1$

$\alpha > 1$ $\gamma > 1$

$$c < \log \log n < \log n < \log^\beta n < n^\epsilon < n < n \log n < n^\alpha < \gamma^n < n!$$

$$2^n \quad ? \quad n!$$

$$\lim_{n \rightarrow +\infty} \frac{2 \times 2 \times 2 \times \cdots \times 2}{n \times (n-1) \times (n-2) \times \cdots \times 1} = 0$$

Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

$\beta > 1$ $0 < \epsilon < 1$

$\alpha > 1$ $\gamma > 1$

$$c < \log \log n < \underline{\log n} < \log^\beta n < \underline{n^\epsilon} < n < n \log n < n^\alpha < \gamma^n < n!$$

$\epsilon = 0.1$

• $f(n) = n \log n$, $g(n) = n^{1.1}$, which one is asymptotically larger?

• A. $f(n)$

• B. $g(n)$

$$\begin{array}{ccccc} & n \log n & ? & n^{1.1} \\ /n & \log n & \leftarrow & n^{0.1} \end{array}$$

Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

$\beta > 1 \quad 0 < \epsilon < 1$

$\alpha > 1 \quad \gamma > 1$

$$c < \log \log n < \log n < \log^\beta n < n^\epsilon < n < n \log n < \underbrace{n^\alpha}_{\alpha = 10} < \underbrace{\gamma^n}_{\gamma = 2} < n!$$

- $f(n) = n^{10}$, $g(n) = 2^n$, which one is asymptotically larger?
 - A. $f(n)$
 - B. $g(n)$

Commonly-used functions

For large enough n , we have (asymptotically):

Constant $c > 0$

$\beta > 1$ $0 < \epsilon < 1$

$\alpha > 1$ $\gamma > 1$

$$c < \log \log n < \underline{\log n} < \log^\beta n < \underline{n^\epsilon} < n < \underline{n \log n} < \underline{n^\alpha} < \gamma^n < n!$$

$\epsilon = 0.5$ $\alpha = 2$

• $f(n) = \frac{n}{\log n}$, $g(n) = \sqrt{n}$, which one is asymptotically larger?

• A. $f(n)$

• B. $g(n)$

	$\frac{n}{\log n}$	?	$\sqrt{n}$
	n	?	$\sqrt{n} \log n$
$\times \log n$	$\sqrt{n}$	>	$\log n$
$/\sqrt{n}$	T^2	>	$T \log T^2 = 2T \log T$
$T = \sqrt{n}$			

(Let's use $\log n$ as $\log_2 n$, and c is a constant)

Simple Rules

- We can **omit constants**
- We can **omit lower order terms**
 - $\Theta(an^2 + bn + c)$ becomes $\Theta(n^2)$
 - $\Theta(c)$ becomes $\Theta(1)$
 - $\Theta(\log_c n)$ becomes $\Theta(\log n)$ (assuming $c > 1$)
 - $\log_c n = \log n / \log c$
 - $\Theta(\log(n^c)) = \Theta(c \log(n))$ becomes $\Theta(\log n)$
 - But $\Theta(\log^2 n) \neq \Theta(\log n)$

Simple Rules

- $f(n) = \Theta(f(n))$
- $\Theta(f(n))\Theta(g(n)) = \Theta(f(n)g(n))$
- $\Theta(f(n)) + \Theta(g(n)) = \Theta(f(n) + g(n))$
- $O(f(n)) - O(g(n)) \neq O(f(n) - g(n))$

$$f(n) = \Theta(n^2), g(n) = \Theta(n^2) \Rightarrow f(n) - g(n) = \Theta(n^2 - n^2) = 0?$$

(No. Consider $f(n) = n^2 + n, g(n) = n^2, f_1(n) - f_2(n) = \Theta(n)$)

Popular Classes of Functions

- **Constant:** $f(n) = \Theta(1)$
- **Logarithmic:** $f(n) = \Theta(\log(n))$
- **Poly-logarithmic:** $f(n) = O(\log^k n)$
- **Sublinear:** $f(n) = o(n)$
- **Linear:** $f(n) = \Theta(n)$
- **Super-linear:** $f(n) = \omega(n)$
- **Quadratic:** $f(n) = \Theta(n^2)$
- **Polynomial:** $f(n) = O(n^k)$
- **Exponential:** $f(n) = O(k^n)$

(But usually more
than polynomial)

Example

1, 10, 100000000

$\log n$, $\log n + \log \log n$

$\log^2 n$, $\log^9 n + 8$

$\log n$, $\sqrt{n}$, $n^{1/5}$

n , $5n + \log n$

n^3 , $n \log n$

n^2 , $3n^2 + n$

$n^3 + 2n^2 + 4$, $4n^5$

2^n

$(k \geq 1 \text{ is a constant})$

Case study: selection sort

- Find the smallest element in the array
- Move it to the front
- For the rest of elements, find the smallest element and repeat

```
for (int i = 0; i < n; i++) {  
    for (int j = i+1; j < n; j++) {  
        if (a[j] < a[i]) swap(a[i], a[j]);  
    }  
}
```

Time complexity

$$(n - 1) + (n - 2) + \dots + 1 = n \times \frac{n - 1}{2} = \Theta(n^2)$$

Why we don't care about constants & lower-order terms?

- Why we only care about the order (main term)?

Quicksort: $O(n \log n)$ time in expectation

```
void qsort(data_type* A, int l, int r)
{
    int i = l, j = r-1;
    data_type x = A[random_number(l, r)];
    do {
        while (A[i] < x) i++;
        while (A[j] > x) j--;
        if (i <= j) {
            swap(A[i], A[j]);
            i++;
            j--;
        }
    } while (i <= j);

    if (i < r - 1) qsort(A, i, r);
    if (l < j) qsort(A, l, j+1);
}
```

Running time in ms:

	Quicksort	Selection sort
10	0.0098	0.0067
100	0.011	0.027
1,000	0.14	1.683
10,000	1.695	286.425
100,000	19.46	13799.5
1,000,000	226.443	1291300
10,000,000	1706.54	

Within 1s

About 20 min

Selection sort: $O(n^2)$ time:

```
for (int i = 0; i < n; i++)
    for (int j = i+1; j < n; j++)
        if (a[i] > a[j]) { swap(A[i], A[j]); }
```

How large a problem you can solve?

- Assume your computer can do each operation in $0.1 \mu\text{s}$ (10^7 operations per second)

Complexity	Max problem size (n)		
	1 second	1 minute	1 hour
$5n$	2,000,000	120,000,000	7,200,000,000
$3n \log_2 n$	172,454	1,003,201	418,961,604
$2n^2$	2,236	17,320	134,164
n^3	215	843	3,301
n^4	56	156	435
2^n	23	29	35

Is RAM model perfect?

$$n = 10^8$$

`for (int i = 0; i < n; i++) A[i] = A[i] + 1;`

0.141072s

`for (int i = 0; i < n; i++) A[i] = (long long)(i)*4323 % n + 1;`

0.580809s

`for (int i = 0; i < n; i++) A[i] = A[(long long)(i)*4323 % n] + 1;`

3.25008s

How long would the other for-loop take?

A. 0.14s

D. 1.0s

B. 0.2s

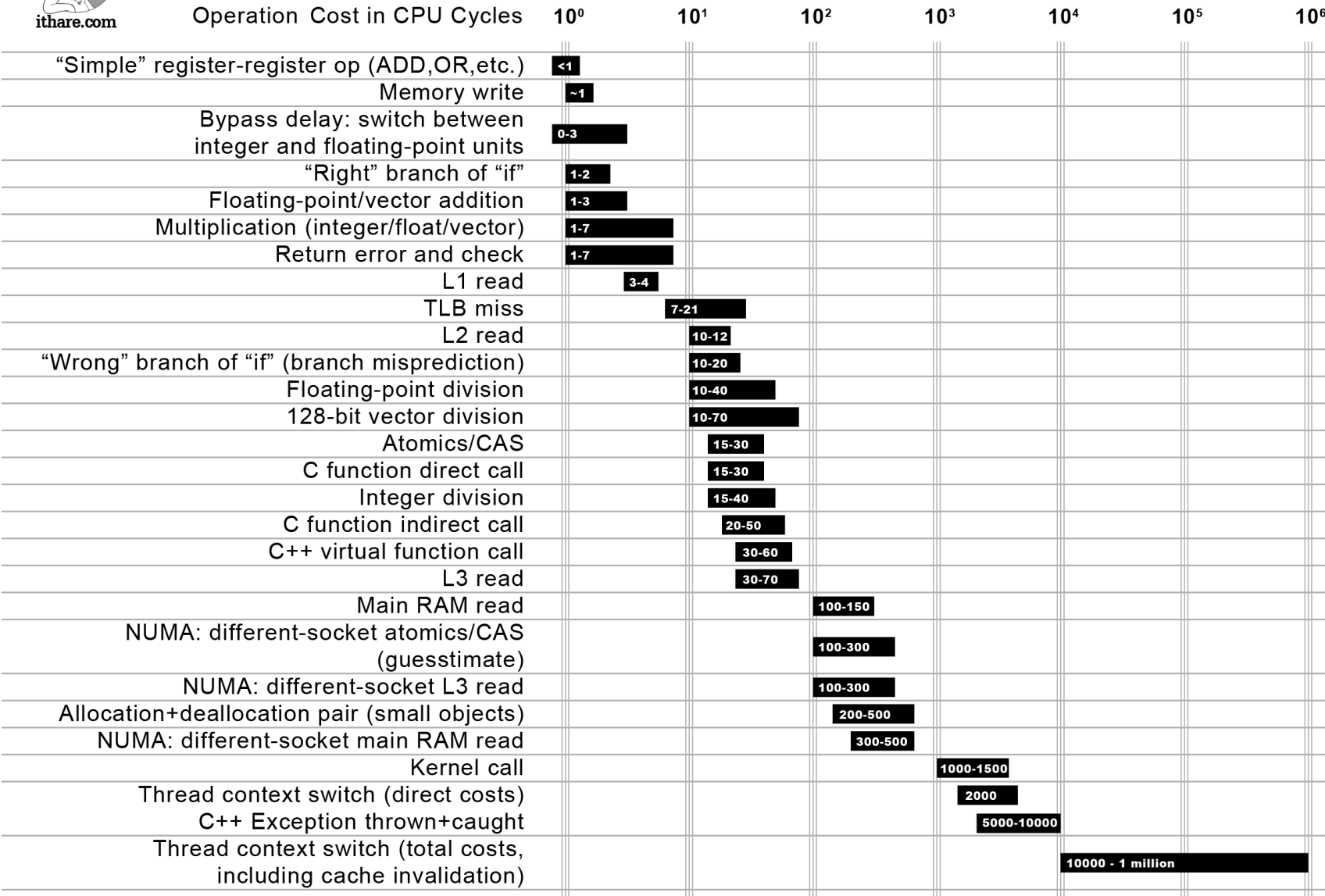
E. 3.0s

C. 0.6s

F. 5.0s



Not all CPU operations are created equal



Distance which light travels while the operation is performed

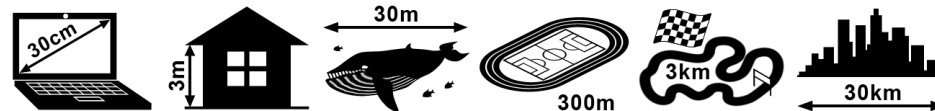


Image from ithare.com:

<http://ithare.com/infographics-operation-costs-in-cpu-clock-cycles/>

Is theory still useful?...

- Other models can explain this!
- (Many other classes will mention this, such as CS142, CS218, CS214)
- Time complexity is still one of the simplest and most elegant way to understand performance in a reasonable effective way.
- More practice in programming will give you a sense of the difference and connections between theory and practice
- Rule of thumb: sequential access is faster than random access
- There are other interesting models!